FMI Layered Standard for DAE (FMI-LS-DAE)

Contents

1. Introduction . . . 2
   1.1. Intent of this Document . . . 2
   1.2. How to read this Document . . . 2
   1.3. Overview . . . 2
2. Common Concepts . . . 2
   Mathematical Notation . . . 2
      DAE representation . . . 2
         Fully-Implicit DAE . . . 3
         Semi-Explicit Index 1 / Hessenberg index 1 DAE . . . 3
         Relationship between fully-implicit and semi-explicit DAE . . . 3
         Semi-Explicit DAE . . . 3
         Mass matrix DAE . . . 4
         Linear time-invariant DAE . . . 4
      Indices . . . 4
         Differential Index . . . 4
         Perturbation Index . . . 4
            Example of Structural Singularity . . . 4
      Hello World DAE Example . . . 6
         HelloWorldDAE - Fully-Implicit DAE . . . 6
         HelloWorldDAE - Semi-Explicit DAE . . . 6
         HelloWorldDAE - Explicit ODE . . . 6
3. Use Cases . . . 7
   3.1. Generic DAE Use Case . . . 7
      3.1.1. Modelica Example CauerLowPassAnalog . . . 7
      3.1.2. Scaling Power Grids Example . . . 9
References . . . 10

This layered standard on top of FMI 3.0 defines how to exchange dynamic models in differential algebraic equation form.

> Although the document refers to version 3.0 of the FMI standard, everything described in this document also applies to all subsequent minor versions. For

> further information on compatibility, see section Versioning and Layered Standards in the FMI 3.0 specification.

Copyright © 2025 The Modelica Association Project FMI.

This document is licensed under the Attribution-ShareAlike 4.0 International license. The code is released under the 2-Clause BSD License. The licenses text can be found in the LICENSE.txt file that accompanies this distribution.

1. Introduction

1.1. Intent of this Document

A differential-algebraic system of equations (DAE) is a system of equations that either contains differential equations and algebraic equations, or is equivalent to such a system [1].

- Avoid the requirement of index reduction inside of FMUs.
  - This may improve accuracy due to better drift handling.
- Avoid local nonlinear equation solvers inside of FMUs.
  - This may improve accuracy and avoid problems with different local and global error tolerances.
- Preserve the sparseness of DAE systems which is lost for the corresponding reduced ODE systems.
  - This may improve the performance by usage of the sparseness.
- Allow connections between constraint FMUs.
  - Connecting reduced ODE FMUs may lead globally to a non-solvable (singular) system but not for unreduced DAE FMUs.

1.2. How to read this Document

1.3. Overview

2. Common Concepts

Mathematical Notation

DAE representation

There are different forms of DAEs.

For simplicity discontinuous states are currently disregarded.

Fully-Implicit DAE

The generic *fully-implicit DAE*, or *implicit DAE*, formulation:

$$F(\dot{z}(t),\ z(t),\ t) = 0$$

where $z(t)$ represents state and additional variables and t is time.

A similar and sometimes more useful representation:

$$F(\dot{x}(t),\ x(t),\ y(t),\ t) = 0$$

with dynamic variables $x(t)$, algebraic variables $y(t)$ and time t.

Note that $x(t)$ is not necessarily the state variables, more on this in section 2.1.1.

Semi-Explicit Index 1 / Hessenberg index 1 DAE

The *semi-explicit index 1*, also called *Hessenberg index 1*, DAE representation

$$\begin{aligned} \dot{x}(t) &= f(x(t),\ y(t),\ t) \\ 0 &= g(x(t),\ y(t),\ t) \end{aligned}$$

with g solvable for y, $det\left(\frac{\partial}{\partial y} g\right) \neq 0$.

Here f is called the *differential equations* and g the *algebraic equations*.

Relationship between fully-implicit and semi-explicit DAE

A fully implicit DAE

$$F(\dot{x}(t),\ x(t),\ t) = 0$$

can always be rewritten as a semi-explicit DAE by introducing a new algebraic variable x'

$$\begin{aligned} \dot{x}(t) &= x'(t) \\ 0 &= F(x'(t),\ x(t),\ t) \end{aligned}$$

Semi-Explicit DAE

The *semi-explicit DAE* representation

$$\begin{aligned} \dot{x}(t) &= f(x(t),\ y(t),\ t) \\ 0 &= g^1(x(t),\ y(t),\ t) \\ 0 &= g^2(x(t),\ y(t),\ t) \\ 0 &= g^3(x(t),\ y(t),\ t) \\ &\vdots \end{aligned}$$

with g^1 solvable for y, $det\left(\frac{\partial}{\partial y} g^1\right) \neq 0$ and $\left\{\frac{d}{dt} g_i^k\right\} \subseteq \{g_i^{k-1}\},\ k>1$.

Mass matrix DAE

$$M\dot{x}(t) = f(x(t), t)$$

where *mass matrix* M is singular.

An equivalent explicit formulation:

$$\begin{bmatrix} M_x & 0 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \dot{x} \\ 0 \end{bmatrix} = \begin{bmatrix} \hat{f}(x(t), y(t), t) \\ g(x(t), y(t), t) \end{bmatrix}$$

See [cui2020mass].

Linear time-invariant DAE

$$E\dot{x}(t) = Ax(t)$$

where matrix E is singular.

Indices

There are multiple, sometimes closely related, definitions of indices for DAE systems. Usually these indices are a measure of how far from an ODE the DAE is.

Differential Index

The minimum number of times a DAE

$$F(\dot{x}(t), x(t), t) = 0$$

has to be differentiated with respect to time t to be able to determine $\dot{x}(t)$ as a function of t and x is called the *differential index*.

See [hairer2006numerical].

Perturbation Index

The *perturbation index* is a measure of the constraints among equations. An index-0 DAE contains neither algebraic loops nor structural singularities. An index-1 DAE contains algebraic loops, but no structural singularities. A DAE with a perturbation index greater than 1, a so-called *higher-index DAE*, contains structural singularities [2].

See [hairer2006numerical].

Example of Structural Singularity

[cellier2006continuous, chapter 7.7 Structural Singularity Elimination] provides a simple electric circuit model that is an higher-index DAE.

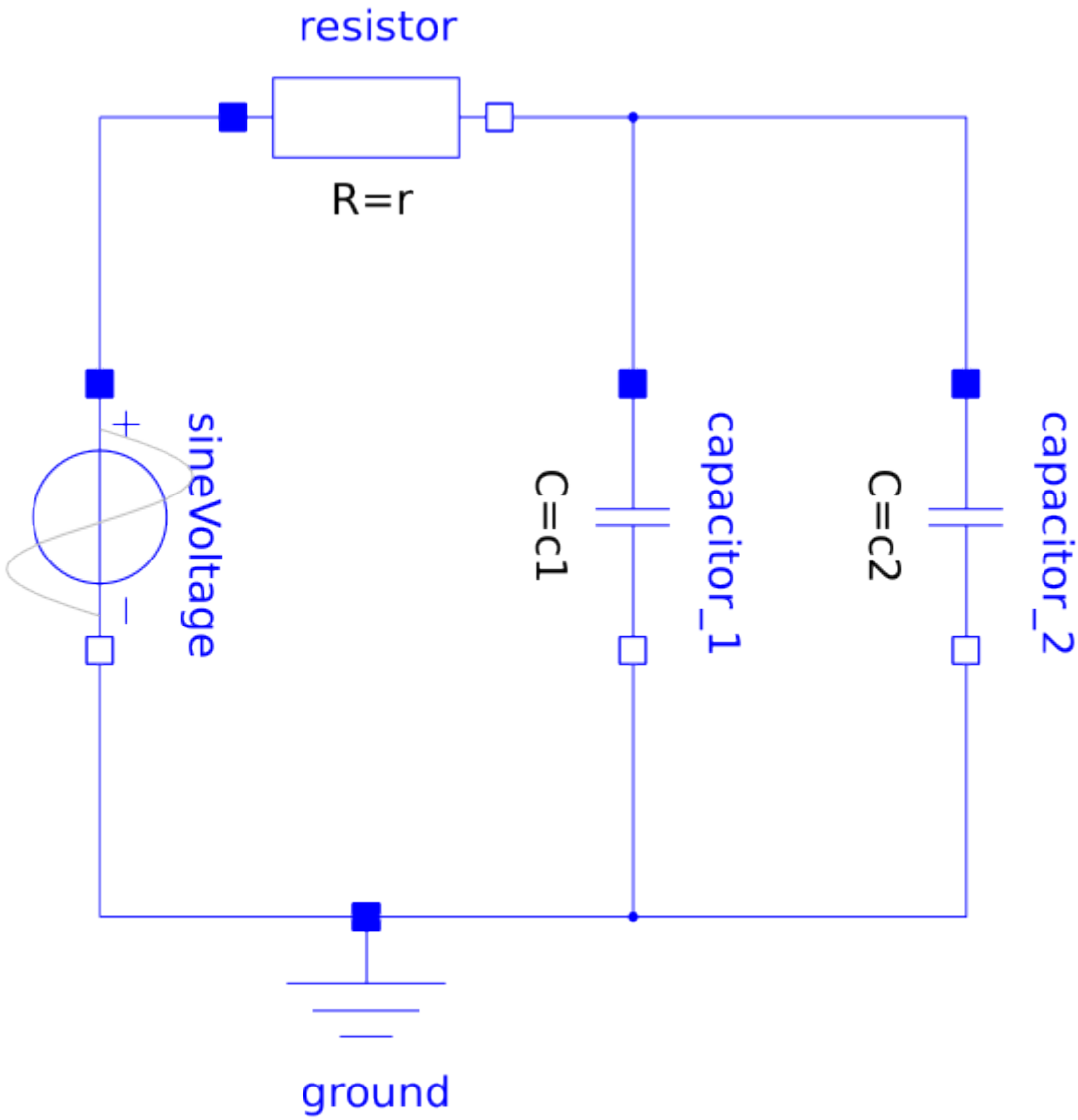


Figure 1. Modelica model of an electric circuit with two capacitors in parallel.

The circuit has two capacitors in parallel and can be represented with equations:

$$
\begin{aligned}
v_0 &= f(t) \\
v_R &= R \cdot i_0 \\
i_1 &= C_1 \cdot \dot{v}_1 \\
i_2 &= C_2 \cdot \dot{v}_2 \\
v_0 &= v_R + v_1 \\
v_2 &= v_1 \\
i_0 &= i_1 + i_2
\end{aligned}
$$

When both capacitive voltages v_1 and v_2 are chosen as state variables equation $v_2 = v_1$ has no unknowns left, so it is a *constraint equation*.

There are different ways to solve this. The modeler could change the causality or exporting tools can try to reduce the perturbation index symbolically [3] to end up with a consistent initialization for the system.

Hello World DAE Example

A simple initial-value problem

$$\begin{aligned} \dot{x} &= y \\ y + 2z &= x \\ y - 3z &= 2x \end{aligned}$$

with variables $x, y, z \in \mathbb{R}$ and start condition

$$\begin{pmatrix} x(t_0) \\ y(t_0) \\ z(t_0) \end{pmatrix} = \begin{pmatrix} x_0 \\ y_0 \\ z_0 \end{pmatrix} \text{and } \dot{x}(t_0) = \dot{x}_0$$

at time t_0. For this specific example a start condition x_0 is sufficient to calculate consistent start values. The independent variable (usually time) t is omitted for simplicity.

HelloWorldDAE - Fully-Implicit DAE

The fully-implicit representation

$$F(\dot{x}, x, t) = 0$$

is given by

$$F : \mathbb{R}^{3 \times 3 \times 1} \to \mathbb{R}^3, \begin{pmatrix} x \\ \dot{x} \\ t \end{pmatrix} \mapsto \begin{pmatrix} \dot{x}_1 - x_2 \\ x_2 + 2x_3 - x_1 \\ x_2 - 3x_3 - 2x_1 \end{pmatrix}$$

HelloWorldDAE - Semi-Explicit DAE

The semi-explicit representation

$$\begin{aligned} \dot{x} &= f(x, y, t) \\ 0 &= g(x, y, t) \end{aligned}$$

is given by

$$\begin{aligned} f &: \mathbb{R}^{1 \times 2 \times 1} \to \mathbb{R}^1, (x, y, t) \mapsto \begin{pmatrix} y_1 \end{pmatrix} \\ g &: \mathbb{R}^{1 \times 2 \times 2} \to \mathbb{R}^1, (x, y, t) \mapsto \begin{pmatrix} y_1 + 2y_2 - x \\ y_1 - 3y_2 - 2x \end{pmatrix} \end{aligned}$$

HelloWorldDAE - Explicit ODE

This simple system could also be transformed into an explicit ODE formulation

$$\dot{x} = \frac{7}{5} x$$

with local variables y and z

$$y = \frac{7}{5}x$$
$$z = -\frac{x}{5}$$

and exported as an ODE ModelExchange FMU.

3. Use Cases

3.1. Generic DAE Use Case

There are a number of possible reasons to export a model as a system of differential-algebraic equations (DAE) instead of a system of ordinary differential equations (ODE):

- The exporting tool doesn't support methods to transform a DAE into ODE representation.
- The time needed to perform symbolic transformation to ODE representation takes a significant amount of time for large systems.
- Large system models are usually characterized by a high degree of sparsity. Some DAE solvers can utilize sparsity to reduce simulation time.
- More efficient simulation of models with large algebraic loops for DAE representation.

See [braun2017solving].

3.1.1. Modelica Example CauerLowPassAnalog

The Modelica model Modelica.Electrical.Analog.Examples.CauerLowPassAnalog from the Modelica Standard Library [4]

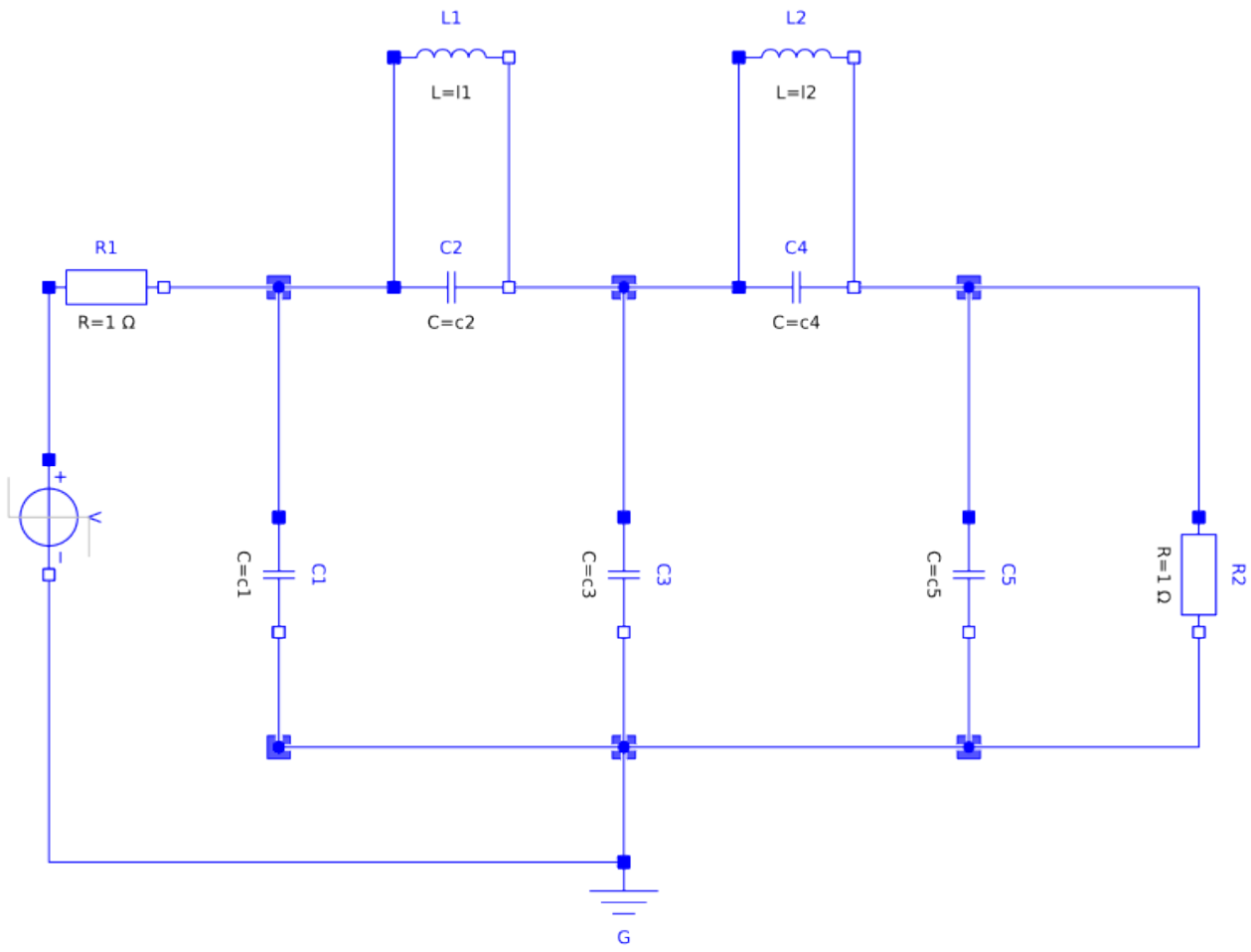


Figure 2. Graphical representation of the Cauer low-pass-filter `Modelica.Electrical.Analog.Examples.CauerLowPassAnalog`.

has structural singularities.

To produce a complete and consistent set of initial conditions for the DAE exporting tools usually use a symbolic index reduction algorithm.

The model has nine state candidates

```
1: L1.v (unit = "V" stateSelect=StateSelect.never ) "Voltage drop of the two pins (=
p.v - n.v)" type: Real
2: L2.v (unit = "V" stateSelect=StateSelect.never ) "Voltage drop of the two pins (=
p.v - n.v)" type: Real
3: C2.v (start = 0.0 unit = "V" ) "Voltage drop of the two pins (= p.v - n.v)" type:
Real
4: C4.v (start = 0.0 unit = "V" ) "Voltage drop of the two pins (= p.v - n.v)" type:
Real
5: L1.i (start = 0.0 unit = "A" fixed = true ) "Current flowing from pin p to pin n"
type: Real
6: L2.i (start = 0.0 unit = "A" fixed = true ) "Current flowing from pin p to pin n"
type: Real
7: C1.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v -
n.v)" type: Real
```

```
8: C5.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v -
n.v)" type: Real
9: C3.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v -
n.v)" type: Real
```

and four constraint equations

```
1: L2.v = C3.v - C5.v
2: L1.v = C1.v - C3.v
3: C4.v = L2.v
4: C2.v = L1.v
```

Modelica tools that want to export an ODE ModelExchange FMU need to reduce the perturbation index to index 1 to end up with an ODE representation of the DAE system.

One option to achieve this is the so called dummy derivative method. From the set of state candidates five state variables are chosen (e.g. `C1.v`,`C3.v`,`C5.v`,`L1.i`,`L2.i`) and the constraint equations

$$\begin{aligned} C2.v &= L1.v \\ L1.v &= C1.v - C3.v \\ C4.v &= L2.v \\ L2.v &= C3.v - C5.v \end{aligned}$$

are differentiated

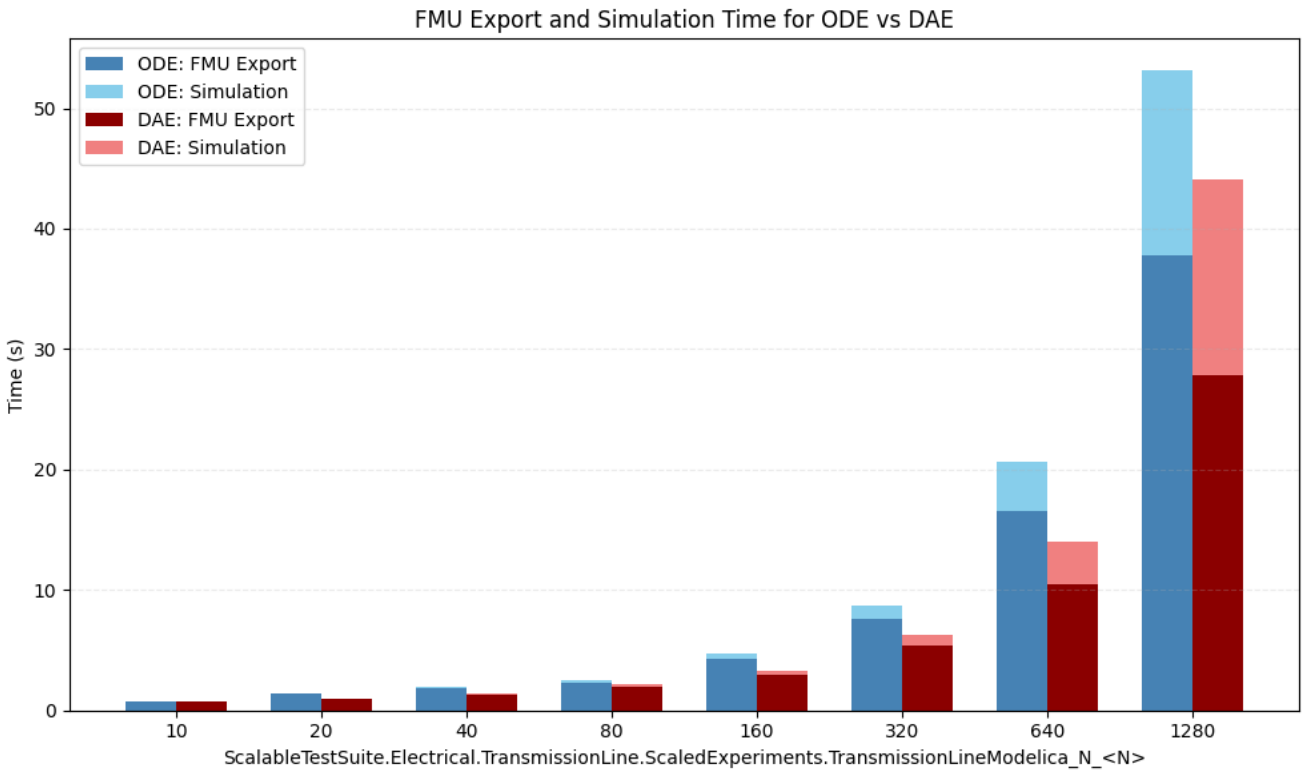
$$\begin{aligned} \frac{d}{dt}C2.v &= \frac{d}{dt}L1.v \\ \frac{d}{dt}L1.v &= \frac{d}{dt}C1.v - \frac{d}{dt}C3.v \\ \frac{d}{dt}C4.v &= \frac{d}{dt}L2.v \\ \frac{d}{dt}L2.v &= \frac{d}{dt}C3.v - \frac{d}{dt}C5.v \end{aligned}$$

which results in four additional dummy states (`$DER.L1.v`, `$DER.L2.v`, `$DER.C2.v`, `$DER.C4.v`). Here Modelica keyword `stateSelect` influences how variables are picked.

3.1.2. Scaling Power Grids Example

For simulation models representing power grids on nation or larger scale traditional ODE solvers may struggle with convergence and performance. Also the time needed to perform symbolic transformation from DAE to ODE is growing with the number of equations.

See for example Modelica models `ScalableTestSuite.Electrical.TransmissionLine.ScaledExperiments.TransmissionLineModelica_N_10` to `ScalableTestSuite.Electrical.TransmissionLine.ScaledExperiments.TransmissionLineModelica_N_1280` from ScalableTestSuite [5].



The simulation was performed using `OpenModelica v1.27-dev-92` with the following settings:

- DAE export: `--daeMode`
- Method: Sundials IDA [6]

Since OpenModelica does not support FMI 3.0 or FMI-LS-DAE at the time of writing, the export and simulation times were measured using OpenModelica's default C export and simulation.

References

- [cui2020mass] Cui, Hantao, Fangxing Li, and Joe H. Chow. "Mass-matrix differential-algebraic equation formulation for transient stability simulation." arXiv preprint arXiv:2008.03883, 2020.
- [cellier2006continuous] Cellier, François E., and Ernesto Kofman. "Continuous system simulation". Boston, MA: Springer US, 2006.
- [hairer2006numerical] Hairer, Ernst, Christian Lubich, and Michel Roche. "The numerical solution of differential-algebraic systems by Runge-Kutta methods". Vol. 1409. Springer, 2006.
- [cellier1993automated] Cellier, Francois E., and Hilding Elmqvist. "Automated formula manipulation supports object-oriented continuous-system modeling". IEEE Control Systems Magazine 13.2 (1993): 28-38.
- [braun2017solving] Willi Braun, Francesco Casella, and Bernhard Bachmann. "Solving Large-Scale Modelica Models: New Approaches and Experimental Results Using OpenModelica". *Linköping Electronic Conference Proceedings (Print)*, pp. 557-563. Linköping University Electronic Press, 2017. https://re.public.polimi.it/bitstream/11311/1065357/1/2017-BraunCasellaBachmann.pdf

- [casella2015simulation] Casella, Francesco. "Simulation of Large-Scale Models in Modelica: State of the Art and Future Perspectives". Proceedings of the 11th International Modelica Conference, pp. 459-468, 2015. https://dx.doi.org/10.3384/ecp15118459

[1] Source: https://en.wikipedia.org/wiki/Differential-algebraic_system_of_equations

[2] From [cellier2006continuous], p. 285.

[3] e.g. by using Pantelides algorithm, see [cellier1993automated]

[4] Modelica Standard Library version 4.1.0

[5] https://github.com/casella/ScalableTestSuite, see also [casella2015simulation]

[6] https://computing.llnl.gov/projects/sundials/ida